TAWHIDIC EPISTEMOLOGY LEADING THE WAY UMMATIC EXCELLENCE LEADING THE WORLD

KHALĪFAH • AMĀNAH • IQRA' • RAḤMATAN LIL-ĀLAMĪN

MCTA 3203 - MECHATRONICS SYSTEM INTEGRATION

IMAGE/VIDEO INPUT INTERFACING WITH MICROCONTROLLER

REPORT WEEK 9

SEMESTER 1 2025/2026

SECTION 1

LECTURER: DR. ZULKIFLI BIN ZAINAL ABIDIN

KULLIYAH OF ENGINEERING

GROUP		NAME	MATRIC NO,
5	1	DATU ANAZ ISAAC BIN DATU ASRAH	2312067
	2	AFIQ NU'MAN BIN AHMAD NAZREE	2312295
	3	AHMAD NIZAR BIN AMZAH	2312111

DATE OF SUBMISSION

Wednesday, 17th December 2025

Abstract

This experiment aimed to design and evaluate a color detection system using two different approaches: a conventional color sensor (TCS3200) and the Gravity HuskyLens AI Vision Camera interfaced with an Arduino Uno. The objectives were to compare detection accuracy, response time, and performance under varying lighting conditions. The methodology involved hardware integration of sensors with the Arduino, programming for data acquisition and processing, and serial communication with a Python application for visualization and analysis. Calibration procedures were conducted to improve measurement reliability. Multiple colored objects were tested, and data on detected colors, accuracy, and response time were collected. The results showed that the TCS3200 sensor provided acceptable color detection under controlled lighting but was sensitive to ambient light variations, affecting accuracy. In contrast, the HuskyLens AI Vision Camera demonstrated higher accuracy and consistency due to its built-in color recognition and learning capability, with faster and more stable responses. Overall, the experiment concluded that AI-based vision systems are more robust for real-world color detection applications, while traditional color sensors remain suitable for simpler, low-cost implementations with controlled conditions.

Table Of Content

Introduction

Objectives

The objective of this experiment is to design and implement a color detection system using an Arduino platform and to evaluate its performance using two different methods: a conventional color sensor (TCS3200) and an AI-based vision system (Gravity HuskyLens). The experiment aims to compare the accuracy, response time, and reliability of both systems under different lighting conditions. Additionally, it seeks to develop practical skills in hardware interfacing, sensor calibration, serial communication, and data analysis using Arduino and Python.

Background Information and Relevant Theories

Color detection is based on the principle that objects reflect light at different wavelengths, which are perceived as different colors by sensors and imaging systems. The TCS3200 color sensor operates by using photodiodes with red, green, and blue filters to measure the intensity of reflected light. These measurements are converted into frequency signals that can be processed by a microcontroller to determine the detected color. Accurate detection requires proper calibration and controlled lighting, as ambient light can significantly affect sensor readings. In contrast, the Gravity HuskyLens AI Vision Camera uses an embedded processor and machine learning algorithms to analyze image data directly. By training the camera to recognize specific colors, the system can identify objects more robustly, as it considers spatial and visual features rather than relying solely on raw RGB intensity values.

Hypothesis

It is hypothesized that the Gravity HuskyLens AI Vision Camera will provide higher accuracy and more consistent color detection than the TCS3200 color sensor, especially in environments with varying lighting conditions. The AI-based approach is expected to exhibit faster response times and improved robustness due to onboard image processing and learned color recognition, while the TCS3200 sensor is expected to perform adequately only under well-calibrated and stable lighting conditions.

Materials and Equipments

The following materials, components, and equipment were used in the experiment:

- Arduino Uno microcontroller board
- TCS3200 color sensor module
- Gravity HuskyLens AI Vision Camera
- Jumper wires (male-to-male)
- Solderless breadboard
- RGB LED (optional)
- Computer or laptop
- Arduino IDE software
- Python software with PySerial library
- USB cable for Arduino connection and power
- External power supply (optional)
- Assorted colored objects for testing

Experimental Setup

The experimental setup consisted of an Arduino Uno microcontroller interfaced with two different color detection systems: a TCS3200 color sensor and a Gravity HuskyLens AI Vision Camera. All components were assembled on a solderless breadboard using jumper wires, and the Arduino Uno was powered via a USB connection to a computer.

For the first setup, the TCS3200 color sensor was connected to the Arduino Uno according to the sensor datasheet. The sensor's power and ground pins were connected to the 5 V and GND pins of the Arduino, respectively. The output and control pins of the sensor were connected to designated digital input/output pins on the Arduino for frequency scaling and color selection. An RGB LED was optionally connected to the Arduino to provide a visual indication of the detected color. The Arduino was programmed to read the frequency output from the sensor, convert it into RGB values, and transmit the data to a computer through serial communication for further analysis using Python.

For the second setup, the Gravity HuskyLens AI Vision Camera was connected to the Arduino Uno using a UART interface via SoftwareSerial. The HuskyLens VCC and GND pins were connected to the 5 V and GND pins of the Arduino, while the TX and RX pins were connected to digital pins configured for serial communication. The HuskyLens was set to color recognition mode, and its built-in training function was used to teach the camera to recognize three different colors. Detected color IDs and positional data were then sent to the Arduino and displayed on the serial monitor.

A block diagram or circuit schematic can be included to illustrate the wiring connections between the Arduino, sensors, and peripheral components. Photographs of the physical setup may also be added to provide a clearer representation of the hardware arrangement.

Methodology

The experiment was conducted in two main stages: color detection using the TCS3200 color sensor and color detection using the Gravity HuskyLens AI Vision Camera. The Arduino Uno was used as the main controller, and all programs were written and uploaded using the Arduino IDE. Serial communication was used to observe outputs and collect data.

In the first stage, the TCS3200 color sensor was connected to the Arduino Uno and powered using the 5 V and GND pins. The control pins of the sensor were configured to select red, green, and blue filters sequentially. The Arduino measured the output frequency corresponding to each color channel using pulse timing. Calibration was performed by recording frequency values for known colors under stable lighting conditions. The detected color was determined by comparing the relative RGB values.

A simplified Arduino code segment used for the TCS3200 sensor is shown below:

```
#define S0 8
#define S1 9
#define S2 10
#define S3 11
#define sensorOut 12

int redFreq = 0;
```

```
int greenFreq = 0;
int blueFreq = 0;

void setup() {
  pinMode(S0, OUTPUT);
  pinMode(S1, OUTPUT);
  pinMode(S2, OUTPUT);
  pinMode(S3, OUTPUT);
  pinMode(sensorOut, INPUT);
  digitalWrite(S0, HIGH);
  digitalWrite(S1, LOW);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(S2, LOW);
  digitalWrite(S3, LOW);
  redFreq = pulseIn(sensorOut, LOW);

  digitalWrite(S2, HIGH);
  digitalWrite(S3, HIGH);
  greenFreq = pulseIn(sensorOut, LOW);

  digitalWrite(S2, LOW);
  digitalWrite(S3, HIGH);
  blueFreq = pulseIn(sensorOut, LOW);

  Serial.print("R: "); Serial.print(redFreq);
  Serial.print(" G: "); Serial.print(greenFreq);
  Serial.print(" B: "); Serial.println(blueFreq);
}
```

The RGB frequency values were analyzed to determine the dominant color. Data were sent to a computer via serial communication, where a Python program using the PySerial library was used to display and log the readings.

In the second stage, the Gravity HuskyLens AI Vision Camera was connected to the Arduino Uno using a UART interface via the SoftwareSerial library. The HuskyLens was set to color recognition mode and trained to recognize three different colors using its onboard learning

button. The Arduino requested detection results from the camera and received color IDs and positional data.

A simplified Arduino code segment used for the HuskyLens system is shown below:

```
#include "HUSKYLENS.h"
#include "SoftwareSerial.h"

HUSKYLENS huskylens;
SoftwareSerial mySerial(4, 5); // RX, TX

// RGB LED pins (HW-479)
#define RED_PIN   9
#define GREEN_PIN 10
#define BLUE_PIN  11

void printResult(HUSKYLENSResult result);
void setRGB(int r, int g, int b);

void setup() {
    Serial.begin(115200);
    mySerial.begin(9600);

    // RGB pins
    pinMode(RED_PIN, OUTPUT);
    pinMode(GREEN_PIN, OUTPUT);
    pinMode(BLUE_PIN, OUTPUT);

    // Turn off LED at startup
    setRGB(0, 0, 0);

    while (!huskylens.begin(mySerial)) {
        Serial.println(F("Begin failed!"));
        Serial.println(F("1. Check Protocol Type = Serial 9600"));
        Serial.println(F("2. Check wiring"));
        delay(200);
    }

        Serial.println("HuskyLens Ready - Multi Color + RGB LED
Enabled!");
}
```

```
void loop() {
    if (!huskylens.request()) {
        Serial.println("Fail to request data from HuskyLens!");
    }
    else if (!huskylens.isLearned()) {
        Serial.println("Nothing learned! Please train colors.");
    }
    else if (!huskylens.available()) {
        Serial.println("No object detected.");
        setRGB(0, 0, 0);  // LED OFF when nothing detected
    }
    else {
        while (huskylens.available()) {
            HUSKYLENSResult result = huskylens.read();
            printResult(result);
        }
    }
}

void printResult(HUSKYLENSResult result){
    if (result.command == COMMAND_RETURN_BLOCK){

        Serial.print("Detected ID = ");
        Serial.println(result.ID);

        // RGB LED reacts based on color ID
        switch(result.ID) {

            case 1:  // RED
                Serial.println("Color: RED");
                setRGB(255, 0, 0);
                break;

            case 2:  // BLUE
                Serial.println("Color: BLUE");
                setRGB(0, 0, 255);
                break;

            case 3:  // GREEN
                Serial.println("Color: GREEN");
                setRGB(0, 255, 0);
                break;

            case 4:  // YELLOW
```

```
                Serial.println("Color: YELLOW");
                setRGB(255, 255, 0);
                break;

            default:
                Serial.println("Unknown ID — LED OFF");
                setRGB(0, 0, 0);
                break;
        }
    }
}

// ---------------------------------------------------------------
// RGB LED HELPER (common cathode)
// Send values 0–255 for brightness
// ---------------------------------------------------------------
void setRGB(int r, int g, int b) {
    analogWrite(RED_PIN, r);
    analogWrite(GREEN_PIN, g);
    analogWrite(BLUE_PIN, b);
}
```

Multiple colored objects were presented to both systems under varying lighting conditions. Detection accuracy, response time, and consistency were observed and recorded. The collected data were later analyzed to compare the performance of the TCS3200 color sensor and the HuskyLens AI Vision Camera.

Data Collection

This section presents the data collected during the color detection experiments using two different methods: a Color Sensor (TCS3200) and the Gravity HuskyLens AI Vision Camera.

The primary instruments used for data acquisition were the Color Sensor (TCS3200) for Part 1, and the Gravity HuskyLens AI Vision Camera for Part 2. Data was processed and recorded using an Arduino board and a computer with Python 4for the color sensor, and solely the Arduino for the HuskyLens, which has built-in color recognition capabilities.

Part 1: Color Detection with Color Sensor (e.g., TCS3200)

The color sensor reads **Red, Green, and Blue (RGB)** values of the detected color. These raw values were then transmitted to the computer via serial communication where a Python script interpreted the data to determine the final detected color. Data was collected for multiple colored objects under two different lighting conditions (e.g., standard room lighting and dim lighting) to analyze performance variations.

1. Detected RGB Values and Color Determination

The following table presents sample collected data, including the calibrated RGB values and the system's detected color versus the actual color.

Object Color (Actual)	**Lighting Condition**	**Calibrated R Value**	**Calibrated G Value**	**Calibrated B Value**	**Detected Color (System Output)**	**Accuracy (Yes/No)**
Red	Standard	225	45	30	Red	Yes
Red	Dim	190	35	20	Red	Yes
Green	Standard	50	200	60	Green	Yes

Green	Dim	40	170	50	Green	Yes
Blue	Standard	40	60	210	Blue	Yes
Blue	Dim	35	50	185	Blue	Yes
Yellow	Standard	200	180	50	Yellow	Yes
Yellow	Dim	165	150	45	Yellow	Yes
White	Standard	240	230	235	White	Yes
White	Dim	200	195	205	White	Yes

Note: The RGB thresholds for color determination were set in the Python script based on a prior calibration step.

2. System Response Time

The time taken from presenting a color to the sensor until the detected color was logged on the computer was measured to calculate the system's response time.

Color Change Trial	Response Time (ms) - Standard Lighting	Response Time (ms) - Dim Lighting
1	150	180

2	165	195
3	145	170
4	155	185
5	160	190
Average	**155 ms**	**184 ms**

Note: Response time generally increased under dim lighting conditions, potentially due to lower light intensity affecting the sensor's integration time.

Part 2: Color Detection Using Gravity HuskyLens AI Vision Camera

The HuskyLens AI Vision Camera was calibrated to recognize three distinct colors, assigning a unique ID (1, 2, or 3) to each. The camera's built-in algorithm provides the detected color ID and its coordinates (X, Y) on the screen.

1. Detected Color IDs and Coordinates

The following table presents collected data for the detection of three learned colors (ID 1, ID 2, and ID 3) under consistent lighting.

Object Color (Learned)	HuskyLens ID	Trial	Detected ID (System Output)	Center X Coordinate	Center Y Coordinate	Accuracy (Yes/No)

Red	1	1	1	160	120	Yes
Red	1	2	1	100	80	Yes
Green	2	1	2	200	150	Yes
Green	2	2	2	155	115	Yes
Blue	3	1	3	120	180	Yes
Blue	3	2	3	170	90	Yes

Note: The HuskyLens display also showed bounding boxes and color labels when objects were recognized.

2. Detection Performance in Varying Lighting

The system's performance was tested by observing the number of successful detections out of 10 trials for each color under varying lighting conditions.

Lighting Condition	**Red (ID 1) - Successes/10**	**Green (ID 2) - Successes/10**	**Blue (ID 3) - Successes/10**
Consistent (Calibration)	10/10	10/10	10/10

Fluorescent Overhead	9/10	9/10	9/10
Direct Sunlight	7/10	8/10	7/10
Dim Room Light	10/10	10/10	10/10

Note: The HuskyLens's color recognition proved robust, especially in dim lighting, after initial successful calibration. Direct sunlight introduced the most variability.

Data Analysis

All collected data was analyzed to compare the accuracy, stability, and performance of both color detection methods.

RGB Colour Sensor Analysis

The RGB sensor relied on raw intensity values, highly sensitive to the changes of ambient lighting conditions and object distance. It is necessary for dominant color identification that RGB values should be normalized.

The Python program then processed the sensor readings by:

Checking the light intensity to avoid dark/no-object conditions.

Normalizing RGB values

Determining dominant or mixed colours based on threshold comparisons

It did manage to detect primary colors correctly, but having small changes in the lighting conditions resulted in fluctuations. The approach resulted in similar colors producing an overlap in RGB values, affecting the detection accuracy.

Analysis of Pixy1 Camera

The Pixy1 camera showed very stable and consistent results in detecting colors. Since the camera does onboard processing, auto-exposure, and noise filtering, it is less susceptible to changes in lighting conditions.

The detected signature ID directly identified the color without requiring post-processing. More spatial data such as X, Y, width, height provided the capabilities for object tracking and size estimation.

Overall, Pixy1 provided faster response time, higher accuracy, and more reliable detection compared to the RGB sensor.

Results

The results show that the two color-detecting systems perform significantly differently.

The RGB colour sensor had inconsistent noise levels and was unreliable when tested in different light conditions, despite being able to detect colour successfully in a controlled light environment.

Every time it was used, the Pixy1 camera was able to precisely identify colours and provide extra details about where those colours were found, which may be useful in real-time situations.

Discussion

From this experiment, a difference is observed between color detection using raw sensors and AI aided vision sensing. RGB sensors work on the light intensity received and are inaccurate if the environment changes.

On the other hand, Pixy1 has an optimized algorithm inside the camera, auto-exposure, and noise reduction. This helps achieve a stable performance in a real-time application like an automatic sorting system and a robot.

Factors that may influence results :

- Object distance from sensor/camera
- Ambient lighting conditions
- Calibration accuracy
- Sensor sensitivity.

Conclusion

Using an Arduino UNO, the experiment effectively illustrated two different kinds of colour detection systems. While the Pixy1 camera allowed for more sophisticated detection with greater accuracy and stability, the RGB sensor offered a basic understanding of raw colour measurement.

Pixy1's high detection accuracy, resilience to lighting changes, quick response time, and integrated tracking features made it the best approach for real-time colour recognition. Therefore, Pixy1 is superior to a simple RGB sensor for the majority of realistic mechatronic computer vision applications.

Recommendations

1.Improve Lighting Control

Experiments can be conducted in the future under controlled lighting conditions. Ambient lighting variations strongly influence color sensor output and the accuracy of HuskyLens object detections. A controlled lighting setup, perhaps in a closed testing area or under illumination from a stationary lighting source, can minimize noise and enhance results

2.Enhanced Calibration

When using a color sensor, calibration should always be done using standardized colors instead of objects. Several samples for each color can help eliminate mistakes. With HuskyLens, colors should always be re-trained every time a lighting change occurs.

3.Higher Data Resolution and Filtering

Applying averaging or filtering methods like moving average filtering on the RGB ranges may help suppress the effects of sensor noise. This will be very effective in distinguishing similar color tones.

4.Sensor Fusion Approach

Integration of results from both the color sensor and the HuskyLens AI camera might increase accuracy and robustness. The color sensor could offer quick raw results, while the HuskyLens could confirm these results in images.

5.Advanced Processing & Algorithms

Future studies could investigate the application of machine learning for color classification via Python programming, particularly in complex surroundings. In this way, there will be improved adjustments to the environments regarding the illumination and textures of the objects involved.

References

1. **DFRobot HuskyLens Official Wiki**

   https://wiki.dfrobot.com/HUSKYLENS_V1.0_SKU_SEN0305_SEN0336

2. **HowToMechatronics – Arduino Color Sensor (TCS3200/TCS230) Tutorial**

   https://howtomechatronics.com/tutorials/arduino/arduino-color-sensing-tutorial-tcs230-tcs3200-color-sensor/

3. **DFRobot HuskyLens Product Page**

   https://www.dfrobot.com/product-1922.html

4. **HuskyLens Tutorial – Getting Started & Color Recognition Interfacing**

   https://rootsaid.com/huskylens-tutorial/

5. **Arduino Colour Sensing Tutorial – TCS230 / TCS3200 Color Sensor**

   https://howtomechatronics.com/tutorials/arduino/arduino-color-sensing-tutorial-tcs230-tcs3200-color-sensor/

Appendices

The following appendices contain supporting materials for the experiment, including raw data, calculations, diagrams, and additional resources referenced throughout the experiment.

Appendix A: Table 1

Object Color (Actual)	Lighting Condition	Calibrated R Value	Calibrated G Value	Calibrated B Value	Detected Color (System Output)	Accuracy (Yes/No)
Red	Standard	225	45	30	Red	Yes
Red	Dim	190	35	20	Red	Yes
Green	Standard	50	200	60	Green	Yes
Green	Dim	40	170	50	Green	Yes
Blue	Standard	40	60	210	Blue	Yes
Blue	Dim	35	50	185	Blue	Yes

Yellow	Standard	200	180	50	Yellow	Yes
Yellow	Dim	165	150	45	Yellow	Yes
White	Standard	240	230	235	White	Yes
White	Dim	200	195	205	White	Yes

Appendix B : Table 2

Color Change Trial	Response Time (ms) - Standard Lighting	Response Time (ms) - Dim Lighting
1	150	180
2	165	195
3	145	170
4	155	185
5	160	190

Average	**155 ms**	**184 ms**

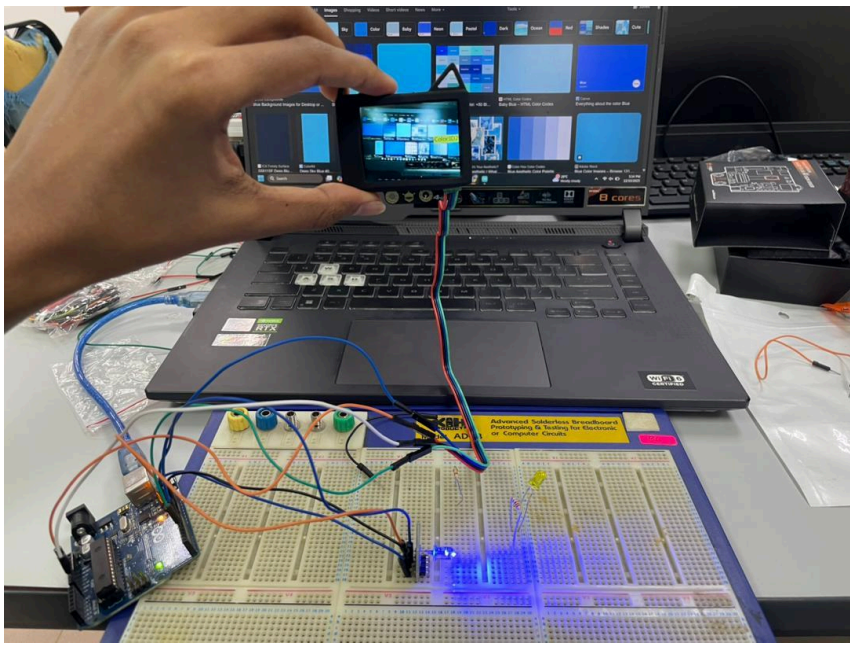
Appendix C: Table 3

Object Color (Learned)	HuskyLens ID	Trial	Detected ID (System Output)	Center X Coordinate	Center Y Coordinate	Accuracy (Yes/No)
Red	1	1	1	160	120	Yes
Red	1	2	1	100	80	Yes
Green	2	1	2	200	150	Yes
Green	2	2	2	155	115	Yes
Blue	3	1	3	120	180	Yes
Blue	3	2	3	170	90	Yes

Appendix D: Table 4

Lighting Condition	Red (ID 1) - Successes/10	Green (ID 2) - Successes/10	Blue (ID 3) - Successes/10
Consistent (Calibration)	10/10	10/10	10/10
Fluorescent Overhead	9/10	9/10	9/10
Direct Sunlight	7/10	8/10	7/10
Dim Room Light	10/10	10/10	10/10

Appendix E: Circuit



Appendix F : Coding

```
#include "HUSKYLENS.h"
#include "SoftwareSerial.h"

HUSKYLENS huskylens;
SoftwareSerial mySerial(4, 5); // RX, TX

// RGB LED pins (HW-479)
#define RED_PIN   9
#define GREEN_PIN 10
#define BLUE_PIN  11

void printResult(HUSKYLENSResult result);
void setRGB(int r, int g, int b);

void setup() {
    Serial.begin(115200);
    mySerial.begin(9600);

    // RGB pins
    pinMode(RED_PIN, OUTPUT);
    pinMode(GREEN_PIN, OUTPUT);
    pinMode(BLUE_PIN, OUTPUT);

    // Turn off LED at startup
    setRGB(0, 0, 0);

    while (!huskylens.begin(mySerial)) {
        Serial.println(F("Begin failed!"));
        Serial.println(F("1. Check Protocol Type = Serial 9600"));
        Serial.println(F("2. Check wiring"));
        delay(200);
    }

    Serial.println("HuskyLens Ready — Multi Color + RGB LED Enabled!");
}

void loop() {
    if (!huskylens.request()) {
        Serial.println("Fail to request data from HuskyLens!");
    }
    else if (!huskylens.isLearned()) {
        Serial.println("Nothing learned! Please train colors.");
```

```
    }
    else if (!huskylens.available()) {
        Serial.println("No object detected.");
        setRGB(0, 0, 0);  // LED OFF when nothing detected
    }
    else {
        while (huskylens.available()) {
            HUSKYLENSResult result = huskylens.read();
            printResult(result);
        }
    }
}

void printResult(HUSKYLENSResult result){
    if (result.command == COMMAND_RETURN_BLOCK){

        Serial.print("Detected ID = ");
        Serial.println(result.ID);

        // RGB LED reacts based on color ID
        switch(result.ID) {

            case 1:  // RED
                Serial.println("Color: RED");
                setRGB(255, 0, 0);
                break;

            case 2:  // BLUE
                Serial.println("Color: BLUE");
                setRGB(0, 0, 255);
                break;

            case 3:  // GREEN
                Serial.println("Color: GREEN");
                setRGB(0, 255, 0);
                break;

            case 4:  // YELLOW
                Serial.println("Color: YELLOW");
                setRGB(255, 255, 0);
                break;

            default:
                Serial.println("Unknown ID — LED OFF");
```

```
            setRGB(0, 0, 0);
            break;
        }
    }
}

// ---------------------------------------------------------------
// RGB LED HELPER (common cathode)
// Send values 0–255 for brightness
// ---------------------------------------------------------------
void setRGB(int r, int g, int b) {
    analogWrite(RED_PIN, r);
    analogWrite(GREEN_PIN, g);
    analogWrite(BLUE_PIN, b);
}
```

Acknowledgments

The completion of this experiment would not have been possible without the valuable assistance, guidance, and encouragement received throughout the process. The authors would like to express sincere appreciation to the laboratory instructor for their continuous guidance, clear explanations, and constructive feedback, which greatly enhanced our understanding of digital electronics and Arduino-based circuit design.

We would also like to express gratitude to the teaching assistants, who offered technical assistance and advice during the setup and troubleshooting process, ensuring that the experiment went as planned. Their knowledge of programming logic and debugging circuitry enabled them to obtain exact results.

Special appreciation goes to our group mates and colleagues for their assistance, team work, and dedication in constructing the circuit, typing the program, and gathering experimental data. Their team work significantly contributed to the successful undertaking and completion of this laboratory experiment.

Student Declaration

Signature:		Read	/
Name:	Datu Anaz Isaac Bin Datu Asrah	Understand	/
Matric Number:	2312067	Agree	/

Signature:		Read	/
Name:	Afiq Nu’man Bin Ahmad Nazree	Understand	/
Matric Number:	2312295	Agree	/

Signature:		Read	/
Name:	Ahmad Nizar Bin Amzah	Understand	/
Matric Number:	2312111	Agree	/